One issue, one review decision

Keep the Copilot cloud agent within the issue

What the cloud agent is

An agent that works **on GitHub, not on your machine**. You hand it a task and come back to a pull request.

```
issue assigned to Copilot
    ↓
agent session starts in a temporary environment
    ↓
agent reads the repo, edits, runs commands and tests
    ↓
agent pushes a branch and opens a draft pull request
    ↓
you review, comment, and merge
```

You can follow the session log while it runs. Review comments on the pull request start another round.

The environment

Each session gets a **fresh, temporary environment**. In an Actions-based session, it is a GitHub Actions runner.

- `.github/workflows/copilot-setup-steps.yml` runs **before** the agent starts. It installs tools and dependencies so the agent can build and test.
- The workflow must have one job named `copilot-setup-steps`, and it must be on the **default branch**.
- Repository instructions (`.github/copilot-instructions.md`) tell the agent how to work.
- A firewall limits network access by default.

Newer sessions can also run in a cloud sandbox, which does not use setup steps. **Check current docs** for which one your plan uses.

Cloud agent vs IDE and CLI agents

	Cloud agent	IDE agent mode / CLI
Runs	On GitHub, in a temporary environment	On your machine
You	Hand off, then review	Watch and steer live
Starts from	An issue or a prompt	Your open workspace
Result	A branch and a pull request	Edits in your working tree
Good for	Well-scoped tasks you can review later	Exploration and tight feedback loops

Same models and similar tools. The difference is where it runs and how you stay involved.

Issue-to-review workflow

```
issue contract
    ↓
repository setup
    ↓
proposed change
    ↓
test output
    ↓
human decision
```

Keep the same issue and acceptance criteria at every step.

The fixed issue

```
Reject blank task titles
```

Allowed files:

- `src/app.js`
- `tests/app.test.js`

Required command:

```
npm test
```

No dependency changes. No unrelated refactor.

Checkpoint 1: Issue contract

Before you assign the issue, a reviewer must be able to answer:

- Which behavior changes?
- Which behavior stays the same?
- Which files may change?
- What proves completion?
- What is out of scope?

Repository instructions

Tell the agent how to work in this repository:

- project shape;
- coding conventions;
- allowed file scope;
- test command;
- prohibited changes.

Instructions support the issue. They don't repair a vague one.

Copilot setup steps

The workflow file lives at:

```
.github/workflows/copilot-setup-steps.yml
```

It has one job named:

```
copilot-setup-steps
```

Review its permissions, commands, and secrets before merging it to the default branch.

Checkpoint 2: Setup

Record:

- baseline test result;
- where the instructions and setup workflow live;
- allowed files;
- reviewer and stop condition.

Stop if you can't explain the baseline result or setup needs a credential.

Live demo: issue to pull request

1. Show the issue and the setup workflow on the default branch.
2. Assign the issue to Copilot.
3. Open the session log. Narrate the plan, commands, and edits.
4. Open the draft pull request. Check the diff and the checks.
5. Comment once to steer, or approve.

If the session is slow, show `proposed-change.diff` and say so.

Start the session

- Assign the fixed issue.
- Watch the session log.
- Steer only within the issue.
- Stop on scope drift.

Checkpoint 3: Proposed change

Inspect before running tests:

- changed files;
- validation behavior;
- response shape;
- valid request behavior;
- dependency and configuration changes.

A small diff can still be wrong.

Expected behavior

```
if (typeof title !== "string" || title.trim() === "") {
  return res.status(400).json({ error: "title is required" });
}
```

The code may differ. The behavior and constraints must not.

Checkpoint 4: Tests

Case	Expected
`""`	HTTP 400
`"  "`	HTTP 400
`" Weekly plan "`	HTTP 201 with preserved response shape

Run the full suite and save the output.

Keep review feedback within the issue

Good:

```
The whitespace case is missing. Add a focused test in tests/app.test.js.
Keep the current response shape and do not change other endpoints.
Run npm test and report the result.
```

Bad:

```
Clean up the API while you are there.
```

Checkpoint 5: Human decision

Choose:

- **approve** when criteria, scope, and tests pass;
- **request changes** when a correction within the issue remains;
- **pause** when test output or ownership is unresolved.

The agent does not make this decision.

Stop conditions

Stop when:

- the requested files expand;
- a new dependency appears;
- the agent asks for credentials;
- tests keep failing with no recovery plan;
- the issue changes mid-run.

Lab handoff

The lab runs **in your training repository on GitHub**. Keep the same issue through all five checkpoints and record your decision.

`proposed-change.diff` is reference material only. Don't apply it.